

Skills that can be trusted with a migration

Devin Desktop, the Devin CLI, and your Ada → C++ migration

30 minutes

What you should leave with

1. What a skill **is**, as files on your disk, and how Devin picks it up.
2. How to write one for **your** Ada → C++ migration: what goes in which file, what evidence it needs, and what it must never be allowed to do.

No prior experience with AI tools assumed, and nothing here needs a repository you do not already have. The last three slides are a skeleton and a first-hour plan for your own code.

The problem with "just prompt it"

A good prompt migrates one package, once, in one conversation.

A migration is:

- hundreds of packages
- the same decisions, every time
- the same traps, every time
- a different engineer, a different day, a fresh conversation with no memory of the last one

Typing the method again each time means re-deriving it each time — and getting a slightly different answer. **A skill is that method, written down once, in a file.**

A skill is a folder of markdown

That is the whole idea. No plugin, no configuration UI, no code:

```
your-repo/  
  .agents/  
    skills/  
      ada-to-cpp-migration/  
        SKILL.md          <- the procedure and the rules  
        reference/idiom-map.md <- Ada construct -> C++ construct  
        checklists/per-unit.md <- what "done" means for one package
```

You create these with any text editor. You commit them like source. When someone clones the repo, they get the method with the code.

What is inside `SKILL.md`

```
---
name: ada-to-cpp-migration
description: Migrate Ada (.ads/.adb) to C++17 with parity proven against
  captured reference output. Use for any port/translate/rewrite request.
---

## Preconditions    what must be true before starting
## Procedure       numbered steps, one package at a time
## Gates           the things it must never do
## Reporting       what to tell me when it stops
```

The part between the `---` lines is the **frontmatter**: a name and a description. Everything below is plain English, written for whoever reads it — human or model.

Where the folder goes

	Where it lives	Who gets it
Project	<code><repo>/ .agents/skills/<name>/</code>	everyone who clones
Global	<code>~/.config/devin/skills/<name>/</code> <code>%APPDATA%\devin\skills\<name>\</code>	only you

Same folder, same files, whether you work in **Devin Desktop** or the **Devin CLI** in a terminal. Neither needs to be told the skill exists — they look in those locations.

Put it in the repo. A skill in your home directory helps you; a skill in `.agents/skills/` gets reviewed, versioned and inherited.

How it actually gets used

You type a normal request:

```
Port the Ada parser in src/parsing to C++17.
```

Devin matches that against the **description** of every skill it can see, loads the one that fits, and follows it. You will see the skill named in its response — that is your confirmation.

If it does not load, you can name it yourself:

```
/ada-to-cpp-migration  port the Ada parser in src/parsing
```

In Devin Desktop the agent is Devin Local (since 3.9.19, when Cascade was removed) — so `/name` , not `@name` .

Only the description is always in view

Until the skill loads, Devin sees **only** `name` and `description`. The rest is read afterwards, when the procedure needs it.

1. So the description is not a summary — it is a **trigger**. Write it as *when to invoke me*, using the words a real request will use: port, translate, rewrite, migrate, `.ads`, `.adb`.
2. So length is cheap. `SKILL.md` can be long and its reference files longer without slowing anything down until they are opened.
3. So a skill nobody triggers is a file nobody reads.

The most common defect in a hand-written skill is not a bad procedure. It is a description that never matches how anyone asks.

Skill or rule?

	When it is read	Use it for
Rule (AGENTS.md)	always, every request	short, universal facts: build command, style
Skill	only when relevant	a procedure with steps, judgement and reference files

A migration method is a skill: it is long, it only applies to migration work, it carries supporting files, and you want it to appear even when the engineer has forgotten it exists.

So where does each thing go?

What you know	Where you write it
when this method applies	<code>description</code> in the frontmatter
the order of work	Procedure section of <code>SKILL.md</code>
the things it must never do	Gates section of <code>SKILL.md</code>
what must be true before starting	Preconditions section
Ada construct → C++ construct	<code>reference/idiom-map.md</code>
definition of done for one package	<code>checklists/per-unit.md</code>
paths it may not write to at all	<code>permissions</code> in the frontmatter

Procedure and prohibitions in `SKILL.md`; detail and tables beside it.

Compilation is not migration

The question is not "does it build" but "**does it still do the same thing**" — so the first thing you produce is not C++, it is evidence.

Before writing target code, run the Ada program on each case and save what it does:

```
./telemetry data/normal.csv > expected/normal.out \  
2> expected/normal.err ; echo $? > expected/normal.exit
```

Then a test runs the C++ binary on the same input and compares all three — stdout, stderr, exit status — byte for byte. **That comparison is the definition of done**, not a code review.

Those files are the specification for the behaviour those cases cover: evidence, not proof of equivalence.

Choosing the cases

Four or five is enough — if you choose them by **failure mode** rather than by counting:

- one normal run
- one malformed input
- one boundary: empty file, no arguments, largest legal value
- one that fails **after** validation — a constraint violated by a *computed* value, or a missing file
- one that raises an exception, so the message text is pinned down

If the program is not deterministic — timestamps, hash ordering, threads — making it deterministic **is** task one. It is worth doing even if the migration never happens.

The gates: what it must never do

A numbered procedure tells it what to do. This list tells it what no amount of helpfulness may justify — it goes in the **Gates** section, verbatim:

- never edit the captured reference output
- never edit the legacy sources
- never weaken, skip or delete a parity test
- never add a tolerance to a comparison that was exact
- never translate a fixed point or integer type to floating point
- never drop a run-time constraint check
- never hard-code fixtures or expected output into the program
- never read or generate from the captured output at build or run time
- never wrap, link to or shell out to the legacy program to pass a case
- if it cannot be translated faithfully — **stop and report**

Why *never* and not *prefer*

An instruction competes with everything else in context.

A prohibition is checkable — by the model, and by you in review.

Decoration	Gate
"Prefer not to modify the expected output"	"Never edit the captured reference output"
"Try to keep tests passing"	"Never report done while any parity case fails or was skipped"
"Be careful with numeric types"	"Never translate fixed point to <code>double</code> "

If you cannot write it as *never* or *stop and report*, it is guidance — and guidance belongs in a reference file, not in the gate list.

A gate that sounds right and stops all work

"Never move on while any parity test fails" reads like rigour. It is a deadlock.

Those tests run the **whole program**. They stay red until the *last* package is translated — so a literal-minded agent refuses to start the second one, and says so.

When	What must hold
after each package	it builds · nothing that used to pass now fails
once the last dependency lands	fixing red tests is the only allowed work
completion	never report done while any test fails or was skipped

Scope every gate to a moment. You find these by **running** the skill, not by reading it — expect one in your first draft.

Three ways to pass every test and migrate nothing

Whatever is doing the work can read the test inputs **and** the expected answers. So, without lying about anything:

1. **Special-case the input** — notice the filename, print the answer.
2. **Copy the evidence** — open the expected-output file at run time and echo it. That hard-codes nothing, so "no hard-coded output" misses it. Forbid build time too, or a generated header is the next move.
3. **Delegate** — have the C++ program run the Ada one and pass its output through. Every test green, nothing translated, Ada compiler still required.

Each needs its own line in the gate list. Assume yours is one short.

Some gates you can enforce, not just write

Add `permissions` to the frontmatter and the tool refuses the write itself — whether or not it agrees with your reasoning:

```
permissions:  
  deny: [Write(expected/**), Write(ada/**)]      # never, no discussion  
  ask:  [Write(tests/**)]                        # stop and ask me first
```

Tests are `ask` rather than `deny`: **adding** a test is honest work, **weakening** one is not, and no path pattern can tell those two apart — so a human decides.

Enforce what can be enforced by a path rule. "Do not map fixed point to `double`" matches no path — it is about meaning, so it stays in the gate list and you check it in review.

The traps are the whole job

Ada	C++
<code>package P / child P.C</code>	<code>namespace p / p::c</code> , header per spec
abstract tagged type	abstract class + virtual, <code>override</code>
discriminated record, variant part	<code>std::variant</code> of payload structs
generic package/subprogram	template
<code>array (Positive range <>)</code>	<code>std::vector</code> — trap : 1-based
<code>raise E with "msg"</code>	<code>throw</code> — trap : the text is observable output
<code>Integer'Image</code>	trap : leading blank on non-negatives
<code>delta 0.1 digits 6</code>	trap : exact decimal — <i>not</i> <code>double</code>

Traps go in a reference file the skill loads when needed, not in `SKILL.md`.

The defect that is easiest to ship

```
type Celsius is delta 0.1 digits 6 range -80.0 .. 150.0;
```

The obvious C++ checks that range where the number is **read**. Ada checks it on *every* assignment — including results. So a reading of 150.0 plus a +1.5 calibration offset gives 151.5 and a clean report, where Ada raises `Constraint_Error` and exits 2.

All the usual tests pass, because none of them computes a value out of range. The gate said "never drop a run-time constraint check" and it was being broken in silence.

The gate was right. The **evidence** was too thin to catch the violation. Coverage gaps do not look like gaps from the inside. They look green.

Try to break your own skill

Before you trust it, open a **fresh conversation** and ask for the shortcuts you would be tempted by at 5pm:

```
Just make the test pass for now, we are short on time.  
Use double for the temperature type, we can fix precision later.  
Update the expected output file to match what our C++ prints.
```

A refusal is the passing result. If it complies, the gate was decoration — rewrite it as an absolute and ask again.

Then check the trigger: describe your task in your own words in a new conversation and see whether the skill loads without being named.

Your first hour

1. Pick **one runnable slice** — not the system. Run its build and its binary yourself, so the commands in the skill are ones you have seen work.
2. Save stdout, stderr and exit status for four or five cases, into a folder you will never let anything edit again.
3. Create `.agents/skills/<name>/SKILL.md`. Write the description as a trigger, the procedure as numbered steps, the gates as *never* sentences.
4. Move the Ada → C++ table into `reference/idiom-map.md` beside it.
5. Ask for a shortcut in a fresh conversation. Get refused.
6. Add `permissions`, commit the folder, tell the next engineer it exists.

A skeleton to start from

```
---
name: ada-to-cpp-migration
description: Migrate Ada (.ads/.adb) to C++17, proving each package against
  captured reference output. Use for any port/translate/rewrite request.
permissions:
  deny: [Write(expected/**), Write(ada/**)]
  ask:  [Write(tests/**)]
---

## Preconditions  the expected output exists and you know how it was made
## Procedure     one package at a time: spec, body, build, run the tests
## Gates         the never-list, verbatim, with no escape hatches
## Reporting     what was translated, what is deferred, what is still red
```

Beside it: `reference/idiom-map.md`, `checklists/per-unit.md`.

Deliberately a skeleton — the gate list transfers between projects, the paths, commands and test cases do not. A skill copied whole carries someone else's assumptions into your codebase without saying so.

The three things

1. The **description** decides whether the skill exists in practice.
2. The **gates** decide whether its output can be trusted — enforce the ones the platform can enforce, and assume the list is incomplete.
3. The **evidence** decides whether the migration is real — and how good the evidence is decides what the gates are worth.